MALWARE ANALYSIS LAB | REVIEW 3: FINAL

NPMShield: Hybrid Static and Dynamic Analysis of Malicious npm Packages

Siri S Gowda | 23BCI0010
B.Tech CSE (Information Security), VIT Vellore
Faculty: Sanmuga Priya M | October 2026

18
real packages analysed

9 of 13
malicious flagged

0 of 5
benign false positives

Contents of Review 3

1. Final methodology
2. Complete implementation
3. Analysis results
4. Malware characteristics
5. Indicators of Compromise (IoCs)
6. Detection results
7. Comparison and results
8. Screenshots
9. Conclusion
10. Future enhancements
11. Complete project demonstration

Problem in one line: new npm malware hides in install scripts and delayed code, so reading it (static) and running it safely (dynamic) are combined.

1. Final methodology

1. Unpack archives on the VM; install nothing
2. Static analyzer scores code and package.json
3. Sandbox runs install scripts, then imports the package
4. Run twice: laptop profile and CI profile
5. Fuse: 0.4 static + 0.6 dynamic; canary floor of 80
6. Gap rule lists packages only the sandbox flags

Scoring

Bands: 0-30 low, 31-60 suspicious, above 60 malicious.

Flagged = score above 30.

Canary rule: an opened fake credential file forces fused score of at least 80.

Gap rule: sandbox above 30 but static not = gap finding.

Every package goes through the same steps in the same order, so the methods can be compared fairly.

1. Final workflow and safety measures

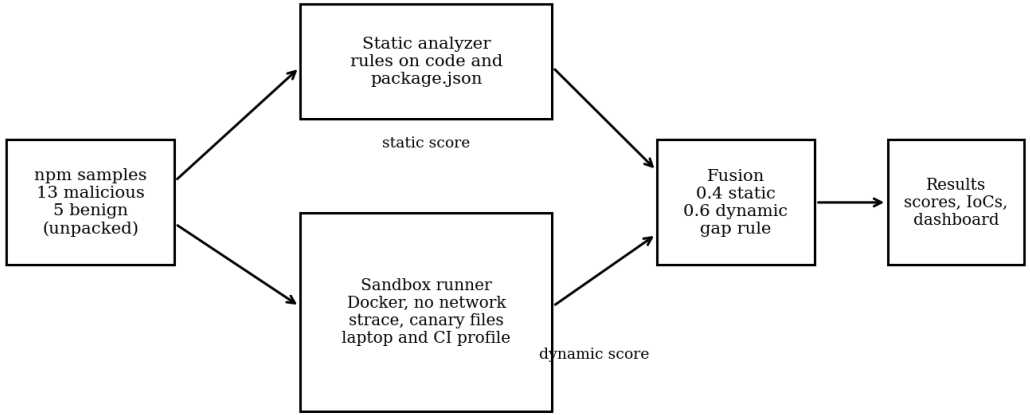


Figure 1: samples go to two analyzers, then fusion, then results

Safety measures

- VM on host-only network; shared clipboard and drag-and-drop off
- Container per run: no network, memory, process and time limits; removed afterwards
- VM snapshot before and after each stage
- Discord webhook token redacted everywhere

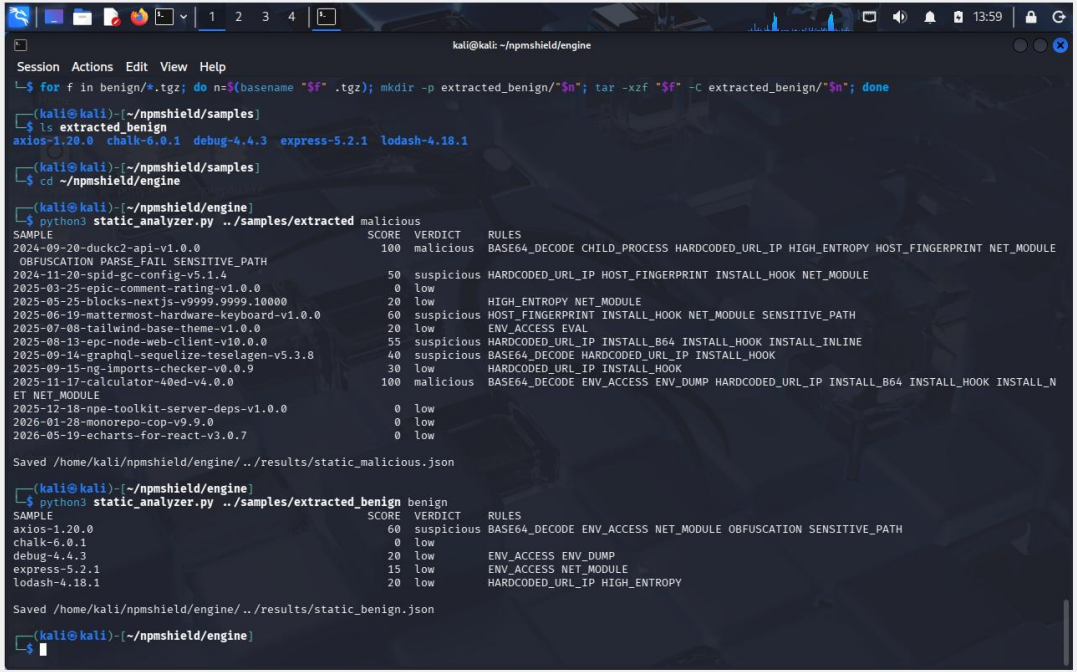
2. Complete implementation

File	Role
static_analyzer.py	Scores code and package.json with rules; extracts static indicators; decodes base64 in install scripts
sandbox_runner.py	Runs each package in Docker under strace, laptop and CI profiles; turns logs into flags and a dynamic score
fusion.py	Fuses scores, applies canary and gap rules, splits indicators by confidence
make_dashboard.py	Reads result files and writes one offline dashboard.html

Sandbox run
Fresh container, no network, 512 MB, 256 processes, 1 CPU. Four canary files planted (.npmrc, SSH key, AWS, git). Install scripts 20 s each, then import for 12 s with timers shortened.

Edge cases handled
Stand-ins for curl, wget and missing modules; top-level import fallback when main is missing; npm's own activity separated as RUNS_NPM; resolver noise removed.

3. Analysis results: static analysis



static_analyzer.py on the malicious and benign sets

6 of 13

malicious flagged statically

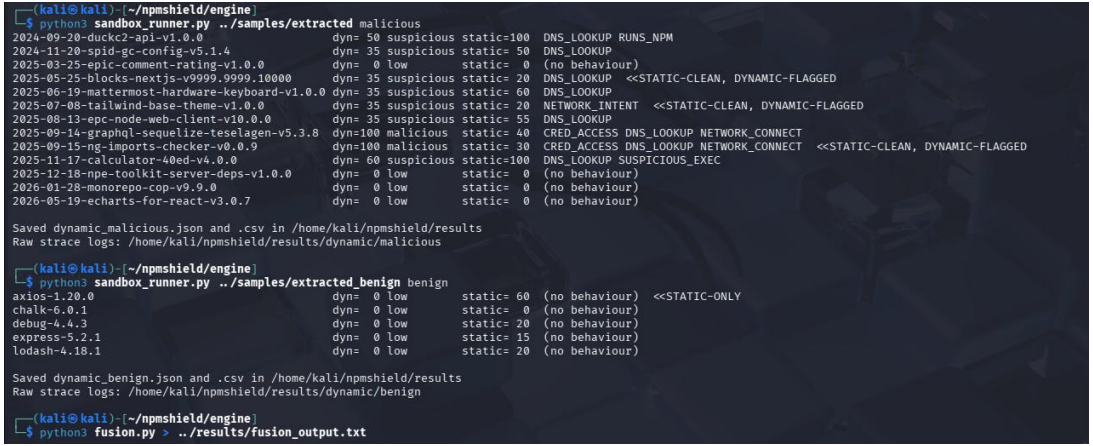
1 of 5

false positive: axios scored 60

4 inert

score 0, checked by hand: placeholders, not misses

3. Analysis results: dynamic analysis



sandbox_runner.py on the malicious and benign sets

Flag	Points
NETWORK_CONNECT	40
DNS_LOOKUP, CRED_ACCESS	35
SUSPICIOUS_EXEC, DROPPED_EXEC	25
ENV_DIFF, RUNS_NPM	15

9 of 13

malicious flagged in sandbox

0 of 5

benign packages flagged

0

ENV_DIFF hits between laptop and CI

3. Analysis results: fused scores

Malicious samples (13)	Static	Dyn.	Fused	Verdict
duckc2-api 1.0.0	100	50	70	malicious
spid-gc-config 5.1.4	50	35	41	suspicious
epic-comment-rating	0	0	0	low (inert)
blocks-nextjs	20	35	29	low (gap)
mattermost-hw-keyboard	60	35	45	suspicious
tailwind-base-theme	20	35	29	low (gap)
epc-node-web-client	55	35	43	suspicious
graphql-sequelize-teselagen	40	100	80	malicious
ng-imports-checker	30	100	80	malicious
calculator-40ed 4.0.0	100	60	76	malicious
npe-toolkit-server-deps	0	0	0	low (inert)
monorepo-cop	0	0	0	low (inert)
echarts-for-react	0	0	0	low (inert)

Benign (5)	Static	Dyn.	Fused
axios 1.20.0	60	0	24
chalk 6.0.1	0	0	0
debug 4.4.3	20	0	8
express 5.2.1	15	0	6
lodash 4.18.1	20	0	8

axios corrected
Static 60 but sandbox 0; fused 24 (low). Fusion removes this false positive.

Canary override
graphql-sequelize-teselagen and ng-imports-checker lifted to 80.

4. Malware characteristics

Credential harvesting
graphql-sequelize-teselagen, ng-imports-checker: at install read .npmrc and .aws/credentials, query npm, GitHub, AWS, try 169.254.169.254:80 (cloud metadata).

Callback beacons
blocks-nextjs, mattermost-hardware-keyboard, calculator-40ed contact unique oastify.com hosts; calculator-40ed sends base64 of /etc/os-release via curl. Some may be research.

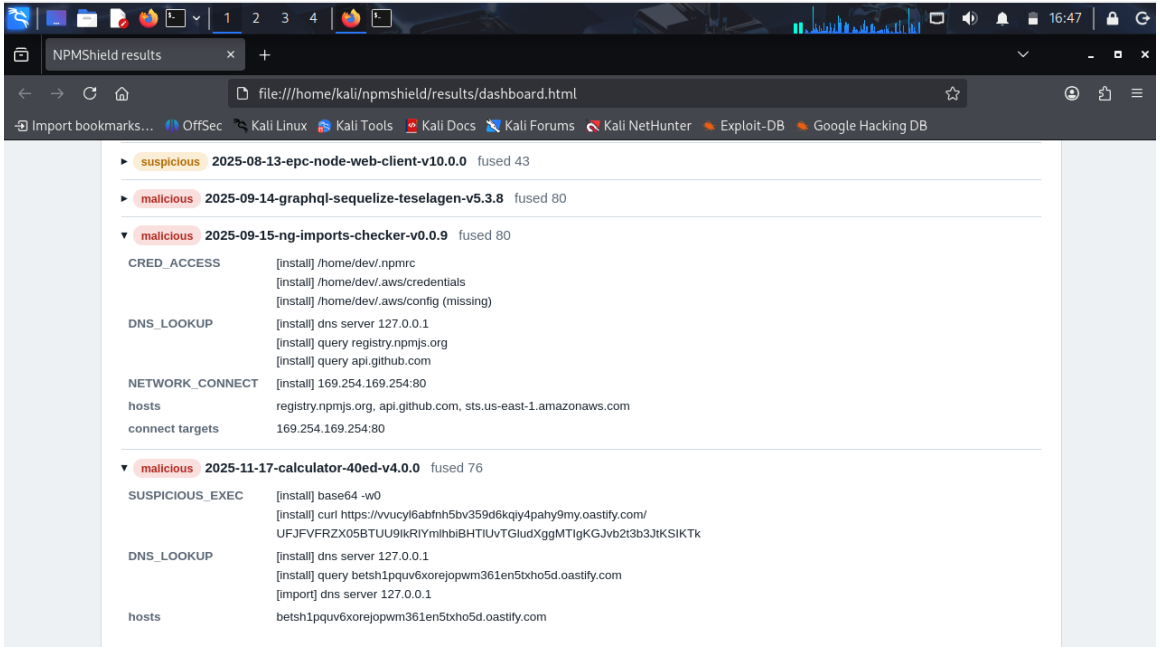
Environment data to hosted endpoint
tailwind-base-theme posts environment variables with axios.post to an ip-check-server vercel.app address (seen via axios stand-in).

Other network behaviour
spid-gc-config: looks up ipinfo.io and discord.com, hardcoded Discord webhook. epc-node-web-client: preinstall base64 decodes to a h1.ci host.

Runtime dependency install
duckc2-api runs npm install for networking packages on import and looks up raw.githubusercontent.com. Purpose not yet determined.

Intent is shown by lookups and connection attempts; what data would be sent is not proven.

4. Sandbox evidence per sample



Dashboard: ng-imports-checker and calculator-40ed

- ng-imports-checker: CRED_ACCESS on .npmrc and .aws/credentials; DNS to registry.npmjs.org and api.github.com; connect to 169.254.169.254:80
- calculator-40ed: SUSPICIOUS_EXEC with base64 and curl to an oastify.com host; second oastify.com lookup
- Static scored ng-imports-checker only 30; the sandbox gave 100

5. Indicators of Compromise: attacker-controlled

Package	Type	Value (defanged)	Basis
spid-gc-config	URL	hxxps://discord[.]com/api/webhooks/1306068586086793297/<redacted>	Hardcoded
blocks-nextjs	Domain	geyybafvoogqfnn3xmadws172y8pwskh[.]oastify[.]com	Observed
mattermost-hw-keyboard	Domain	bp4lqfam2mw2ha36kfd6ehcvemke84wt[.]oastify[.]com	Observed
tailwind-base-theme	URL	hxxps://ip-check-server[.]vercel[.]app/api/ip-check/208	Observed
epc-node-web-client	Domain	wvzo30h2[.]h1[.]ci	Observed, decoded
calculator-40ed	Domain	vvucyl6abfnh5bv359d6kqiy4pahy9my[.]oastify[.]com	Observed
calculator-40ed	Domain	betsh1pquv6xorejopwm361en5txho5d[.]oastify[.]com	Observed

Discord webhook token is redacted. Domains are defanged. Basis: observed in sandbox, decoded from install script, or hardcoded in code.

5. IoCs: legitimate services and low-confidence strings

Package	Services contacted
duckc2-api	raw[.]githubusercontent[.]com
spid-gc-config	discord[.]com, ipinfo[.]io
graphql-sequelize-teselagen	169[.]254[.]169[.]254:80, api[.]github[.]com, registry[.]npmjs[.]org, sts[.]us-east-1[.]amazonaws[.]com
ng-imports-checker	same four targets as above

Do not blocklist
These are real services the samples target (npm, GitHub, AWS, Discord). They show what the malware wants, and blocking them would break normal work.

Low-confidence strings
20 more domains and 20 IPs appear only as text inside package code (doc links, target and proxy lists), for example docs.sequelizejs.com, help.baidu.com, ip-api.com.

None was seen in the sandbox, so they are not reported as indicators. They stay in iocs.csv for manual triage.

6. Detection results

Method	Malicious flagged	Benign FP	Precision	Recall	Accuracy (18)	Accuracy (14)
Static only	6/13 (46%)	1/5 (20%)	6/7 (86%)	46%	10/18 (56%)	10/14 (71%)
Dynamic only	9/13 (69%)	0/5	9/9 (100%)	69%	14/18 (78%)	14/14 (100%)
Fused score	7/13 (54%)	0/5	7/7 (100%)	54%	12/18 (67%)	12/14 (86%)
Fused + gap rule	**9/13 (69%)**	**0/5**	**9/9 (100%)**	**69%**	**14/18 (78%)**	**14/14 (100%)**

9 of 13

fused + gap rule flags (9 of 9 active)

0 of 5

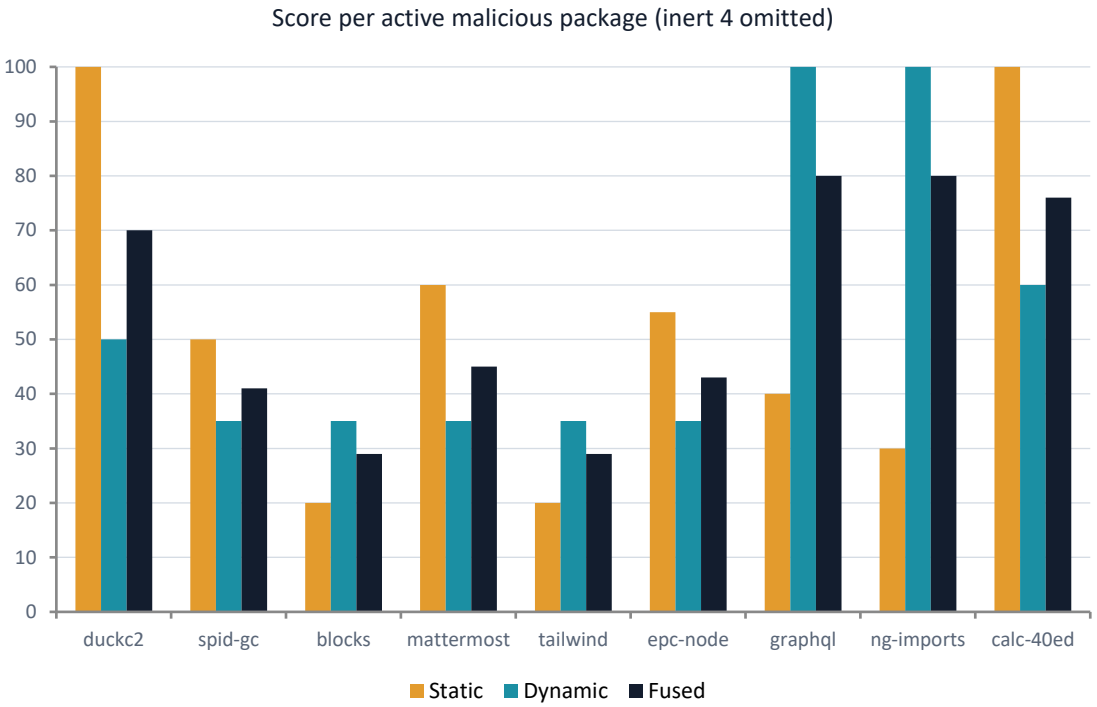
benign false positives after fusion

100%

precision: every flagged package is malicious

Accuracy (14) leaves out the 4 inert packages.

7. Comparison of results



- Sandbox caught 3 packages static rated low: ng-imports-checker 30 vs 100, blocks-nextjs 20 vs 35, tailwind-base-theme 20 vs 35
- axios: static 60, sandbox 0, fused 24, so fusion removed the false positive
- Weighted average alone drops blocks-nextjs and tailwind to 29, so the gap rule is needed: 7/13 becomes 9/13
- Fusion gives one number; the sandbox finds the extra malware. Static needs no setup and runs in seconds

8. Screenshots: fusion and dashboard

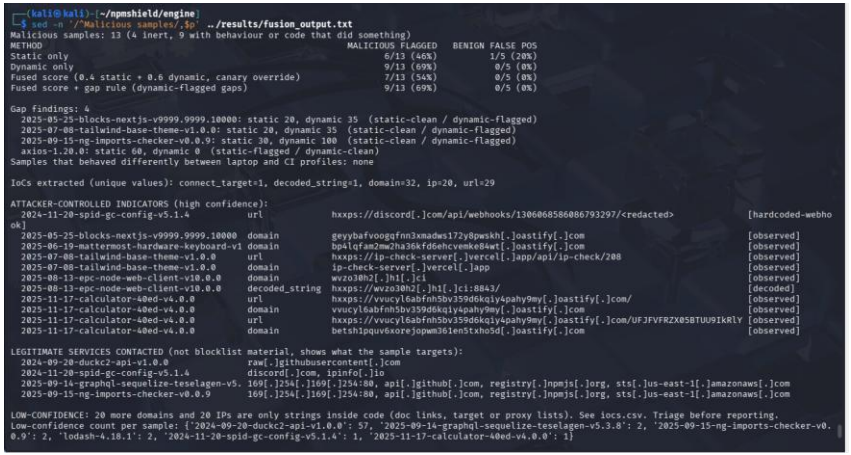


Figure: output of fusion.py (methods, gap findings, indicators)

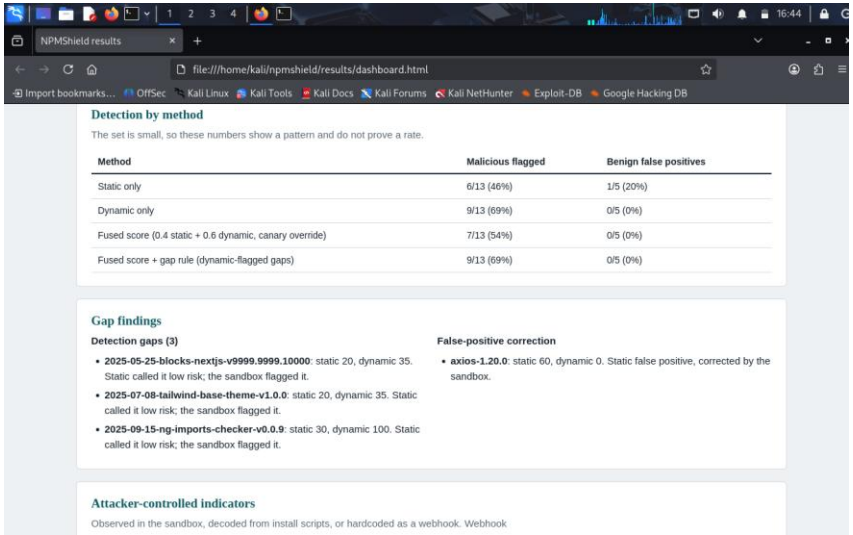
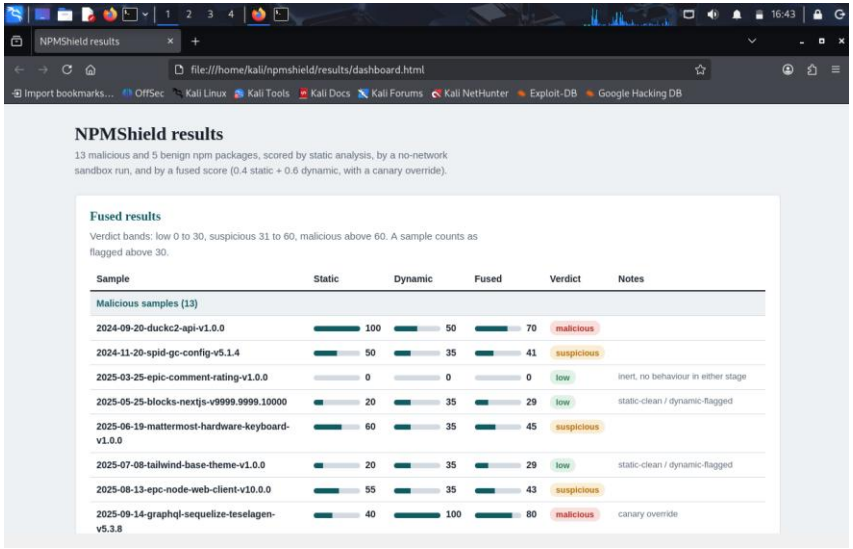


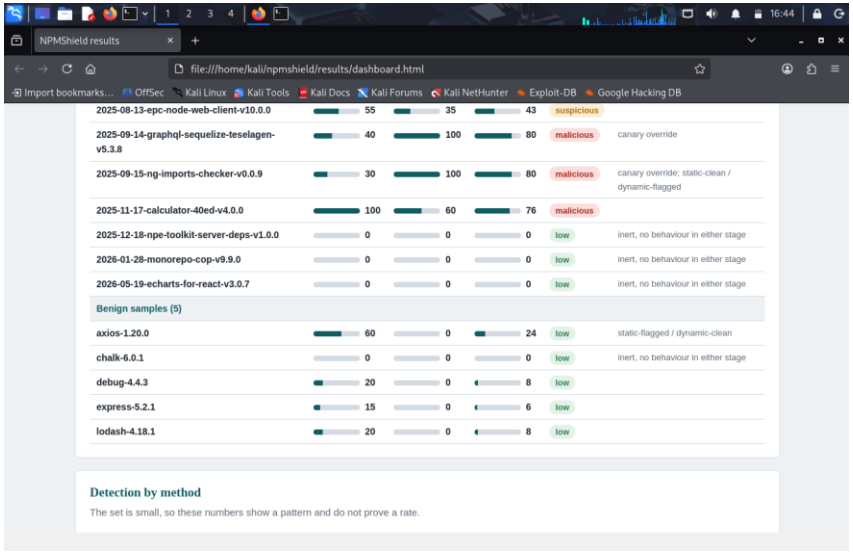
Figure: dashboard detection by method and gap findings

More screenshots: static analyzer (slide 6), sandbox runner (slide 7), sandbox evidence (slide 10).

8. Screenshots: dashboard fused results

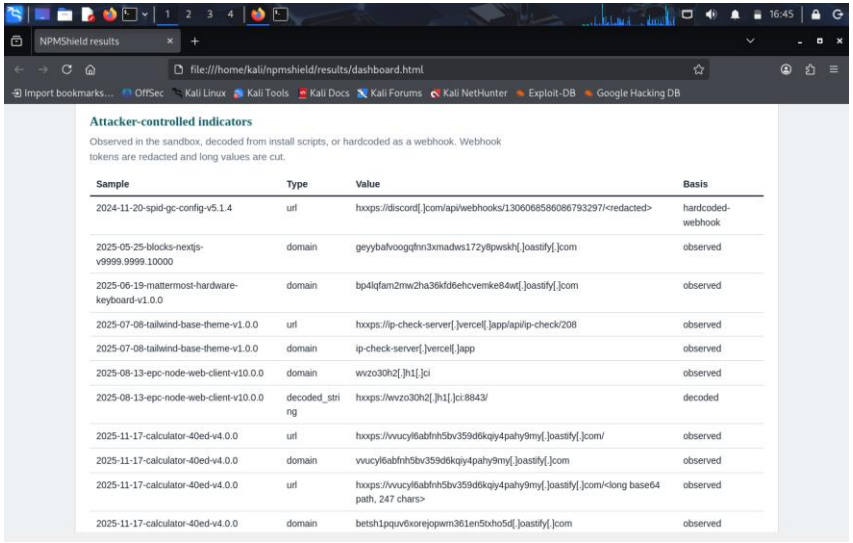


Dashboard: fused results table with static, dynamic and fused bars and verdicts

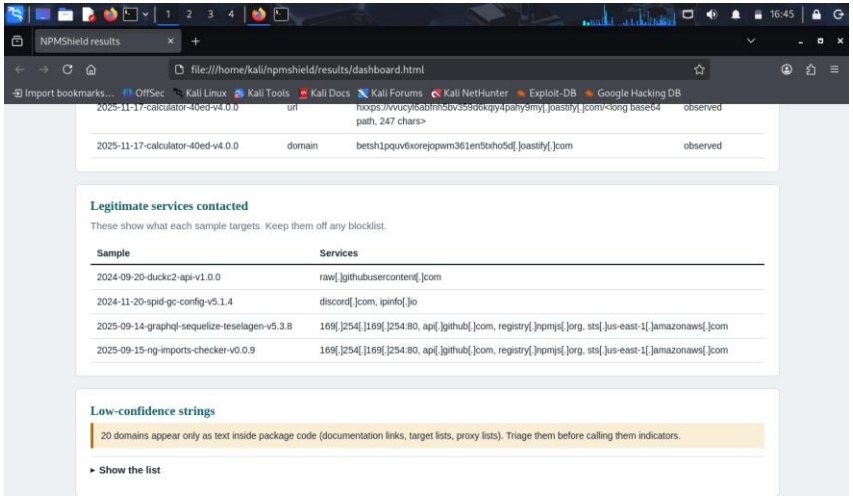


Dashboard: benign samples; axios static 60, dynamic 0, fused 24

8. Screenshots: indicators of compromise



Dashboard: attacker-controlled indicators (webhook token redacted)



Dashboard: legitimate services contacted (not for blocklists)

8. Screenshots: sandbox runner output

```
┌──(kali㉿kali)-[~/npmshield/engine]
└─$ python3 sandbox_runner.py ../samples/extracted malicious
2024-09-20-duckc2-api-v1.0.0                dyn= 50 suspicious static=100  DNS_LOOKUP RUNS_NPM
2024-11-20-spid-gc-config-v5.1.4            dyn= 35 suspicious static= 50  DNS_LOOKUP
2025-03-25-epic-comment-rating-v1.0.0       dyn=  0 low        static=  0  (no behaviour)
2025-05-25-blocks-nextjs-v9999.9999.10000   dyn= 35 suspicious static= 20  DNS_LOOKUP  <<STATIC-CLEAN, DYNAMIC-FLAGGED
2025-06-19-mattermost-hardware-keyboard-v1.0.0 dyn= 35 suspicious static= 60  DNS_LOOKUP
2025-07-08-tailwind-base-theme-v1.0.0       dyn= 35 suspicious static= 20  NETWORK_INTENT  <<STATIC-CLEAN, DYNAMIC-FLAGGED
2025-08-13-epc-node-web-client-v10.0.0      dyn= 35 suspicious static= 55  DNS_LOOKUP
2025-09-14-graphql-sequelize-teselagen-v5.3.8 dyn=100 malicious  static= 40  CRED_ACCESS DNS_LOOKUP NETWORK_CONNECT
2025-09-15-ng-imports-checker-v0.0.9        dyn=100 malicious  static= 30  CRED_ACCESS DNS_LOOKUP NETWORK_CONNECT  <<STATIC-CLEAN, DYNAMIC-FLAGGED
2025-11-17-calculator-40ed-v4.0.0           dyn= 60 suspicious static=100  DNS_LOOKUP SUSPICIOUS_EXEC
2025-12-18-npe-toolkit-server-deps-v1.0.0   dyn=  0 low        static=  0  (no behaviour)
2026-01-28-monorepo-cop-v9.9.0              dyn=  0 low        static=  0  (no behaviour)
2026-05-19-echarts-for-react-v3.0.7         dyn=  0 low        static=  0  (no behaviour)

Saved dynamic_malicious.json and .csv in /home/kali/npmshield/results
Raw strace logs: /home/kali/npmshield/results/dynamic/malicious

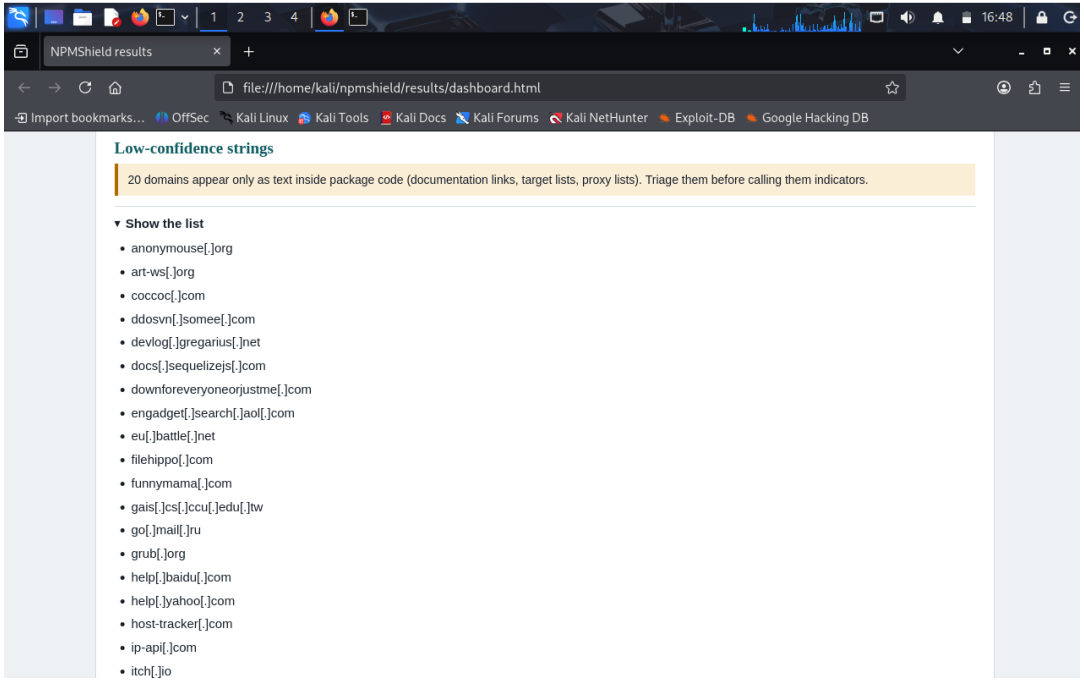
┌──(kali㉿kali)-[~/npmshield/engine]
└─$ python3 sandbox_runner.py ../samples/extracted_benign benign
axios-1.20.0                                dyn=  0 low        static= 60  (no behaviour)  <<STATIC-ONLY
chalk-6.0.1                                 dyn=  0 low        static=  0  (no behaviour)
debug-4.4.3                                 dyn=  0 low        static= 20  (no behaviour)
express-5.2.1                               dyn=  0 low        static= 15  (no behaviour)
lodash-4.18.1                               dyn=  0 low        static= 20  (no behaviour)

Saved dynamic_benign.json and .csv in /home/kali/npmshield/results
Raw strace logs: /home/kali/npmshield/results/dynamic/benign

┌──(kali㉿kali)-[~/npmshield/engine]
└─$ python3 fusion.py > ../results/fusion_output.txt
```

sandbox_runner.py: DYNAMIC-FLAGGED rows show packages static analysis rated low; axios is STATIC-ONLY

8. Screenshots: low-confidence strings



Dashboard: low-confidence list expanded

Why separate

These domains appear only as text in package code. None was seen in the sandbox, so they are kept for manual triage and not reported as indicators.

Limitations

- Small set: 13 malicious and 5 benign; percentages show a pattern, not a real detection rate
- Four malicious packages are inert, so they cannot test either method
- Packages ran without dependencies; express and axios probably stopped at import, so their zero scores are weak evidence
- A lookup or connection attempt shows intent, not what data would be sent
- Only JavaScript timers are shortened; date-triggered payloads would not fire
- No laptop vs CI difference seen, so that design part was built but not shown to matter

9. Conclusion

We built a working pipeline that scores npm packages by reading them, running them offline in a container, and fusing the two.

Result
On 18 packages, 9 of 13 malicious flagged and no benign package flagged by the sandbox.

Key finding
Three packages static called low risk were caught by the sandbox; one opened planted credential files and tried the cloud metadata service.

Honest reading
Small set, four empty samples, no dependencies. Treat the numbers as a first test.

The approach is worth using; static and dynamic analysis complement each other.

10. Future enhancements

01
Larger, more varied sample set including known-active packages

02
Offline cache of common dependencies so packages import fully

03
Change the system date in the container to test date-triggered payloads

04
Record network packets with the tcpdump capture already in the image

05
Resolve open items: what duckc2-api downloads, what spid-gc-config posts to Discord

06
Web interface that takes a package name and returns a report

11. Complete project demonstration

1. **Safe setup** Kali VM, host-only network, docker images, unpacked samples
2. **Live sandbox run** python3 sandbox_runner.py ../samples/extracted malicious calculator-40ed
3. **Fusion** python3 fusion.py: scores, gap findings, indicators
4. **Dashboard** python3 make_dashboard.py, then open dashboard.html in Firefox
5. **Evidence rows** ng-imports-checker, tailwind-base-theme, calculator-40ed

Thank you

Questions? Siri S Gowda | 23BCI0010 | Faculty: Sanmuga Priya M